DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Assignment Report

A Java-based Campus Lost and Found Application

Using Java Swing and JDBC

Field	Details
Department	B.Tech Computer Science and BioScience
Programme	Gokulapriya B(192571008)
Course Code & Name	CSA0902 - Programming in JAVA
Academic Year/Batch	2025
Faculty Name	Dr.Sarala V
Assignment Title	A Java-based Campus Lost and Found Application Using Java Swing and JDBC
Date of Issue	02/02/2026
Date of Submission	02/02/2026
Maximum Marks	100
Course Outcomes	CO4 - GUI design using AWT/Swing, event handling, layout managers CO5 - JDBC connectivity and SQL CRUD operations
Bloom's Taxonomy Level	L3 - Apply | L4 - Analyse
SDG Mapping	SDG 11 - Sustainable Cities and Communities SDG 12 - Responsible Consumption and Production

1.Problem Statement

Design and develop a Campus Lost-and-Found Management System using Java Swing and JDBC. The application must implement GUI components for reporting, updating, and removing lost or found item records, and must maintain the information in a relational database. The system directly supports SDG 11 (Sustainable Cities and Communities) by improving the efficiency and accessibility of a shared campus service, and SDG 12 (Responsible Consumption and Production) by reducing the replacement of items that could otherwise be recovered and reused.

2.Objectives

The main objective of this project is to develop a Java-based Campus Lost and Found Management System that provides an organized and efficient way to manage lost and found item records within a campus. The system is designed using Java Swing and JDBC, combining a user-friendly graphical interface with a relational database. It aims to simplify the process of reporting lost or found items and maintaining their details in a centralized database. The application allows users or operators to add new records, view existing records, update item information, search for specific records, and delete records when required. Another important objective is to ensure that incorrect or incomplete information is not stored by validating mandatory fields, contact numbers, and dates before database operations. The system also maintains the claim status of each item as UNCLAIMED or CLAIMED. Overall, the project aims to improve the accessibility, reliability, and efficiency of the campus lost-and-found service while supporting responsible reuse of recoverable items.

3. Requirement Understanding & System Analysis

2.1 User Roles

- Reporting User - any student/staff member who lost an item or found an item on campus and wants to log it.
- Administrator / Front-desk Operator - the person operating the application at the lost-and-found desk who adds, verifies, updates the claim status of, and deletes records.

2.2 Functional Requirements

ID	Requirement	Mapped CO
FR-1	Capture a new LOST or FOUND item report through a Swing form.	CO4
FR-2	Persist every report as a row in a MySQL database table via JDBC.	CO5
FR-3	Display all current records in a tabular (J-Table) view.	CO4
FR-4	Allow an existing record to be selected, edited, and updated.	CO4 / CO5
FR-5	Allow an existing record to be permanently deleted, with confirmation.	CO4 / CO5
FR-6	Allow keyword and status-based searching/filtering of records.	CO5
FR-7	Validate mandatory fields, contact number format, and date format before any database write.	CO4
FR-8	Track a claim status (UNCLAIMED / CLAIMED) per item.	CO5

2.3 Non-Functional Requirements

- Usability - a single-window layout so the operator never loses context while switching between reporting, browsing, and searching.
- Reliability - every database call is wrapped in try-with-resources and SQL Exception handling so a failure never crashes the GUI.
- Data integrity - a primary key (item_id) and NOT NULL constraints prevent duplicate or incomplete records.

2.4 Application Workflow

On launch, the application loads the JDBC driver, silently creates the database schema if it does not already exist, and loads all existing records into the table. The operator can then Add a new report, select a row to Update or Delete it, or use the Search panel to filter the visible records. Every write operation refreshes the JTable directly from the database so the on-screen view is always consistent with persisted data.

4. System Design

Process Flowchart

The flowchart below traces the complete control flow of the application, from startup through every CRUD and search operation, including the validation and exception-handling branches required by the rubric.

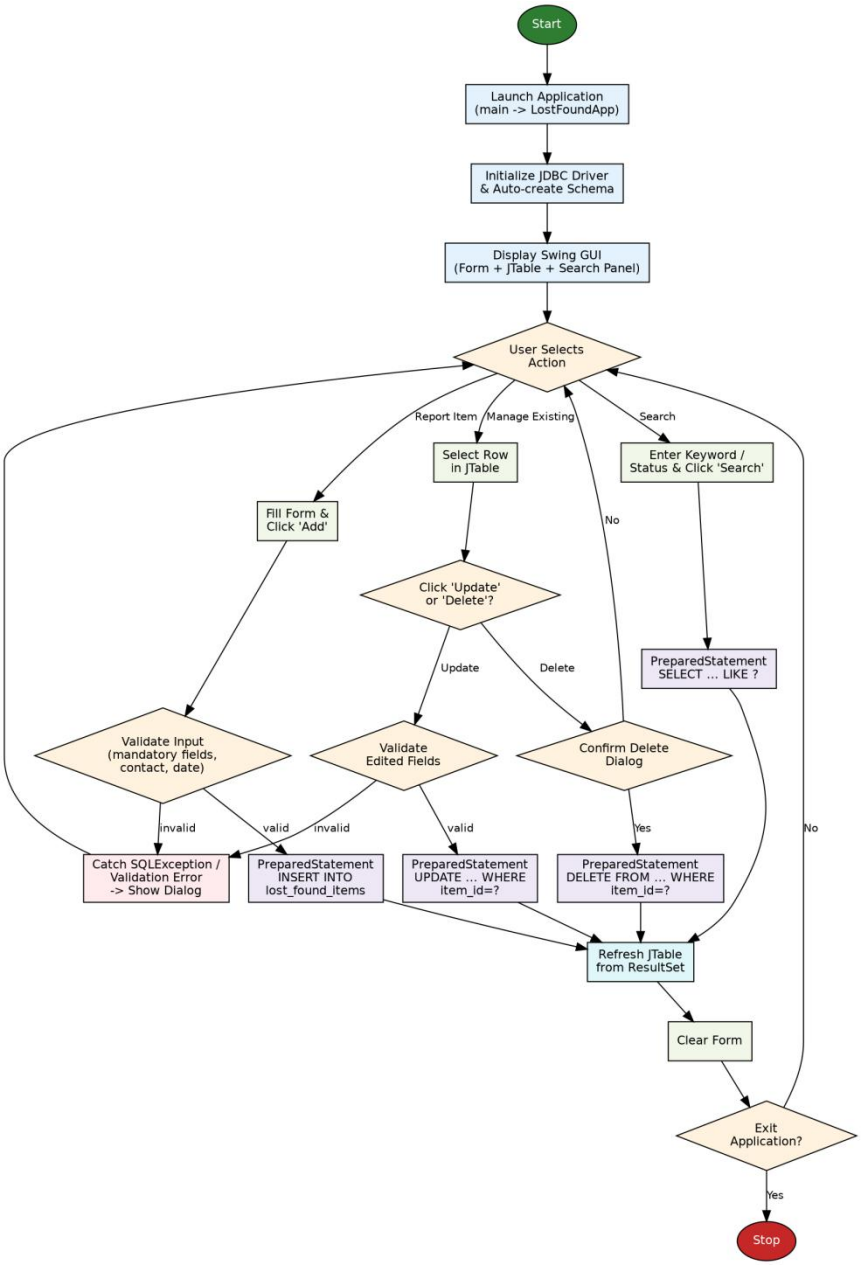

Figure 1: Process flowchart of the Campus Lost & Found Management System

3.2 Database Design

A single, normalized table is sufficient for this domain since each lost/found report is a self-contained record with no repeating groups. item_id is the primary key and auto-increments, guaranteeing a unique identifier for every UPDATE and DELETE operation.

lost_found_items	
PK item_id	INT, AUTO_INCREMENT
item_name	VARCHAR(100) NOT NULL
category	VARCHAR(50) NOT NULL
status	ENUM('LOST','FOUND') NOT NULL
location	VARCHAR(100) NOT NULL
date_reported	DATE NOT NULL
reporter_name	VARCHAR(80) NOT NULL
contact_number	VARCHAR(15) NOT NULL
description	VARCHAR(255)
item_status	VARCHAR(20) DEFAULT 'UNCLAIMED'

Figure 2: Table structure of lost_found_items

GUI Design (CO4)

The interface uses a Border Layout at the top level (form NORTH, J-Table CENTER, search bar SOUTH) and a Grid-Bag Layout inside the form panel so labels and fields align into a clean two-column grid regardless of window re-sizing. J-Combo Box restricts Category, Status, and Item Status to a fixed vocabulary to keep the data clean, while J-Table with a non-editable Default Table Model gives a read-only, sort-able view of the database.

Swing / AWT Component	Purpose in this Application
J-Frame	Top-level application window (Lost Found App).
JPanel + GridBagLayout	Aligns the 9 form fields into a two-column grid.
JTextField	Free-text input: item name, location, date, reporter, contact, description.
JComboBox	Constrained input: category, status (LOST/FOUND), item status.
JTable + DefaultTableModel	Displays and refreshes all database records.
J-Button	Add, Update, Delete, Clear, Search, Show All actions.
JOptionPane	Success confirmations, validation errors, and delete confirmation dialog.
JScrollPane	Scroll able wrapper around the J-Table for large record sets.

5.Pseudo-code

The pseudo-code below describes every event-driven module of the application at an implementation-independent level, before mapping to the actual Java/Swing/JDBC code in Section 5.

```
PSEUDOCODE - Campus Lost & Found Management System
====================================================

MODULE MAIN
  START
      LOAD JDBC Driver (com.mysql.cj.jdbc.Driver)
      CALL initializeSchema()          // CREATE TABLE IF NOT EXISTS
      CREATE MainWindow (JFrame)
      BUILD FormPanel, TablePanel, SearchPanel
      CALL refreshTable()              // populate JTable from DB
      DISPLAY MainWindow
  END

MODULE ADD_ITEM (triggered by "Add" button click)
  READ itemName, category, status, location, dateText,
       reporterName, contact, description, itemStatus FROM form
  IF any mandatory field is empty THEN
      SHOW validation dialog
      RETURN
  END IF
  IF contact does NOT match 10-digit pattern THEN
      SHOW validation dialog
      RETURN
  END IF
  IF dateText is NOT a valid yyyy-MM-dd date THEN
      SHOW validation dialog
      RETURN
  END IF
  TRY
      OPEN JDBC connection
      PREPARE "INSERT INTO lost_found_items (...) VALUES (?,?,?,?,?,?,?,?,?)"
      BIND parameters
      EXECUTE update
      SHOW success dialog
      CLEAR form
      CALL refreshTable()
  CATCH SQLException e
      SHOW database-error dialog WITH e.getMessage()
  FINALLY
      CLOSE connection (try-with-resources)
  END TRY

MODULE UPDATE_ITEM (triggered by "Update" button click)
  IF no row selected in JTable THEN
      SHOW "select a row" warning
      RETURN
  END IF
  VALIDATE form fields (same rules as ADD_ITEM)
  TRY
      PREPARE "UPDATE lost_found_items SET ... WHERE item_id = ?"
      BIND parameters INCLUDING selectedItemId
      EXECUTE update
      SHOW success dialog
      CLEAR form
      CALL refreshTable()
  CATCH SQLException e
      SHOW database-error dialog
  END TRY

MODULE DELETE_ITEM (triggered by "Delete" button click)
  IF no row selected THEN
      SHOW "select a row" warning
      RETURN
  END IF
```

```
  ASK confirmation ("Delete permanently?")
  IF user confirms THEN
      TRY
          PREPARE "DELETE FROM lost_found_items WHERE item_id = ?"
          EXECUTE update
          SHOW success dialog
          CALL refreshTable()
      CATCH SQLException e
          SHOW database-error dialog
      END TRY
  END IF

MODULE SEARCH_ITEMS (triggered by "Search" button click)
  READ keyword, statusFilter FROM search panel
  PREPARE "SELECT * FROM lost_found_items
           WHERE (item_name LIKE ? OR location LIKE ? OR category LIKE ?)
           [AND status = ?]"
  BIND "%keyword%" to LIKE parameters
  IF statusFilter != "ALL" THEN BIND statusFilter END IF
  EXECUTE query -> ResultSet rs
  CLEAR table model
  WHILE rs.next()
      APPEND row TO JTable model
  END WHILE

MODULE REFRESH_TABLE
  EXECUTE "SELECT * FROM lost_found_items ORDER BY item_id DESC"
  CLEAR table model
  FOR EACH record IN ResultSet
      APPEND record TO JTable model
  END FOR

MODULE ROW_SELECTED (JTable ListSelectionListener)
  READ selected row values
  POPULATE form fields with row values
  STORE selected item_id for later UPDATE / DELETE
```

6.Implementation

Technology Stack

Layer	Technology
Language	Java (JDK 17+)
GUI Toolkit	Java Swing (javax.swing) and AWT (java.awt)
Database Connectivity	JDBC (java.sql), PreparedStatement API
Database	MySQL 8.x
Driver	mysql-connector-j (com.mysql.cj.jdbc.Driver)
Build / Run	javac / java, or any IDE (Eclipse / IntelliJ / NetBeans)

Project Structure

```
LostAndFoundApp/
 |-- src/main/java/lostfound/
 |     |-- DBConnection.java     (JDBC connection manager + schema init)
 |     |-- LostFoundItem.java    (POJO / data model)
 |     |-- ItemDAO.java          (Data Access Object - all SQL / CRUD)
 |     |-- LostFoundApp.java     (Swing GUI + event handling, main())
 |-- sql/
 |     |-- schema.sql            (CREATE TABLE + sample seed data)
 |-- lib/
```

```
|     |-- mysql-connector-j-8.x.x.jar
|-- README.md
```

Database Connectivity Layer (CO5)

Db-connection centralizes the JDBC URL, credentials, and driver loading so the rest of the application never touches connection details directly. initialize-schema() runs a CREATE TABLE IF NOT EXISTS statement on startup, which makes the submission self-installing for evaluation - the grader only needs an empty MySQL server.

- **DBConnection.java**

```
package lostfound;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;

/**
 * Centralised JDBC connection manager for the Campus Lost & Found system.
 * Uses a single-connection-per-request model with try-with-resources at the
 * call site, and creates the schema automatically on first run so the
 * application is self-installing for evaluation purposes.
 */
public final class DBConnection {

    private        static        final        String        URL                                                                    =
"jdbc:mysql://localhost:3306/campus_lost_found?useSSL=false&serverTimezone=UTC";
    private static final String USER     = "root";
    private static final String PASSWORD = "root";

    private DBConnection() { /* utility class */ }

    static {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            throw  new  ExceptionInInitializerError("MySQL  JDBC  Driver  not  found  on  classpath:  "  +
e.getMessage());
        }
    }

    /** Opens and returns a new JDBC connection. Caller is responsible for closing it. */
    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }

    /** Creates the required table the first time the application is run. */
    public static void initializeSchema() {
        String ddl = "CREATE TABLE IF NOT EXISTS lost_found_items (" +
                "item_id INT AUTO_INCREMENT PRIMARY KEY," +
                "item_name VARCHAR(100) NOT NULL," +
                "category VARCHAR(50) NOT NULL," +
                "status ENUM('LOST','FOUND') NOT NULL," +
                "location VARCHAR(100) NOT NULL," +
                "date_reported DATE NOT NULL," +
                "reporter_name VARCHAR(80) NOT NULL," +
                "contact_number VARCHAR(15) NOT NULL," +
                "description VARCHAR(255)," +
                "item_status VARCHAR(20) NOT NULL DEFAULT 'UNCLAIMED'" +
                ")";
        try (Connection con = getConnection();
             Statement st = con.createStatement()) {
```

```
            st.executeUpdate(ddl);
        } catch (SQLException e) {
            System.err.println("Schema initialisation failed: " + e.getMessage());
        }
    }
}
```

Data Model

▸ LostFoundItem.java

```
package lostfound;

import java.sql.Date;

/** Plain data model mapping one row of the lost_found_items table. */
public class LostFoundItem {

    private int itemId;
    private String itemName;
    private String category;
    private String status;         // LOST / FOUND
    private String location;
    private Date dateReported;
    private String reporterName;
    private String contactNumber;
    private String description;
    private String itemStatus;     // UNCLAIMED / CLAIMED

    public LostFoundItem() { }

    public LostFoundItem(String itemName, String category, String status, String location,
                         Date dateReported, String reporterName, String contactNumber,
                         String description, String itemStatus) {
        this.itemName = itemName;
        this.category = category;
        this.status = status;
        this.location = location;
        this.dateReported = dateReported;
        this.reporterName = reporterName;
        this.contactNumber = contactNumber;
        this.description = description;
        this.itemStatus = itemStatus;
    }

    // --- Getters and setters ---
    public int getItemId() { return itemId; }
    public void setItemId(int itemId) { this.itemId = itemId; }

    public String getItemName() { return itemName; }
    public void setItemName(String itemName) { this.itemName = itemName; }

    public String getCategory() { return category; }
    public void setCategory(String category) { this.category = category; }

    public String getStatus() { return status; }
    public void setStatus(String status) { this.status = status; }

    public String getLocation() { return location; }
    public void setLocation(String location) { this.location = location; }

    public Date getDateReported() { return dateReported; }
    public void setDateReported(Date dateReported) { this.dateReported = dateReported; }

    public String getReporterName() { return reporterName; }
    public void setReporterName(String reporterName) { this.reporterName = reporterName; }
```

```
    public String getContactNumber() { return contactNumber; }
    public void setContactNumber(String contactNumber) { this.contactNumber = contactNumber; }

    public String getDescription() { return description; }
    public void setDescription(String description) { this.description = description; }

    public String getItemStatus() { return itemStatus; }
    public void setItemStatus(String itemStatus) { this.itemStatus = itemStatus; }
}
```

Data Access Object - CRUD Operations (CO5)

Item DAO is the only class that contains SQL. Every method uses Prepared Statement, which both parameterize user input against SQL-injection and lets the driver correctly bind typed values (e.g. java.sql.Date). Connections, statements and result sets are all opened in try-with-resources blocks so they are closed even when an exception propagates.

- **ItemDAO.java**

GUI and Event Handling (CO4)

LostFoundApp wires every Swing component to its DAO call through lambda-based ActionListeners (this::onAdd, this::onUpdate, …). A ListSelectionListener on the JTable repopulates the form whenever a row is clicked, enabling in-place editing. readFormOrNull() centralises input validation (mandatory fields, a 10-digit contact number, and a parseable yyyy-MM-dd date) and returns null after showing a JOptionPane error, so Add/Update simply abort when validation fails.

- **LostFoundApp.java**

```
package lostfound;

import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.sql.Date;
import java.sql.SQLException;
import java.time.LocalDate;
import java.util.List;
import java.util.regex.Pattern;

/**
 * Campus Lost & Found Management System — main GUI.
 * Demonstrates AWT/Swing layout managers, event handling, JTable data binding,
 * input validation and JDBC-backed CRUD (CO4 + CO5).
 */
public class LostFoundApp extends JFrame {

    private final JTextField txtItemName   = new JTextField(15);
    private final JComboBox<String> cbCategory = new JComboBox<>(new String[]{"Electronics", "Documents",
"Accessories", "Bags", "Books", "Other"});
    private final JComboBox<String> cbStatus   = new JComboBox<>(new String[]{"LOST", "FOUND"});
    private final JTextField txtLocation   = new JTextField(15);
    private final JTextField txtDate       = new JTextField(10); // yyyy-MM-dd
    private final JTextField txtReporter   = new JTextField(15);
    private final JTextField txtContact    = new JTextField(12);
    private final JTextField txtDescription = new JTextField(20);
    private final JComboBox<String> cbItemStatus = new JComboBox<>(new String[]{"UNCLAIMED", "CLAIMED"});

    private final JTextField txtSearch = new JTextField(15);
    private final JComboBox<String> cbSearchStatus = new JComboBox<>(new String[]{"ALL", "LOST", "FOUND"});
```

```
    private final DefaultTableModel tableModel =
            new DefaultTableModel(new Object[]{"ID", "Item", "Category", "Status", "Location",
                    "Date", "Reporter", "Contact", "Description", "Item Status"}, 0) {
                @Override public boolean isCellEditable(int row, int col) { return false; }
            };
    private final JTable table = new JTable(tableModel);

    private final ItemDAO dao = new ItemDAO();
    private Integer selectedItemId = null; // null => Add mode, non-null => Update mode

    public LostFoundApp() {
        super("Campus Lost & Found Management System");
        DBConnection.initializeSchema();
        buildUI();
        loadTableData();
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(1000, 640);
        setLocationRelativeTo(null);
    }

    // ------------------------------------------------------------------ UI --
    private void buildUI() {
        setLayout(new BorderLayout(8, 8));
        add(buildFormPanel(), BorderLayout.NORTH);
        add(new JScrollPane(table), BorderLayout.CENTER);
        add(buildSearchPanel(), BorderLayout.SOUTH);
        table.getSelectionModel().addListSelectionListener(e -> {
            if        (!e.getValueIsAdjusting()        &&        table.getSelectedRow()        !=        -1)
populateFormFromRow(table.getSelectedRow());
        });
    }

    private JPanel buildFormPanel() {
        JPanel panel = new JPanel(new GridBagLayout());
        panel.setBorder(BorderFactory.createTitledBorder("Report / Update Item"));
        GridBagConstraints gc = new GridBagConstraints();
        gc.insets = new Insets(4, 6, 4, 6);
        gc.fill = GridBagConstraints.HORIZONTAL;

        addField(panel, gc, 0, 0, "Item Name:", txtItemName);
        addField(panel, gc, 2, 0, "Category:", cbCategory);
        addField(panel, gc, 0, 1, "Status:", cbStatus);
        addField(panel, gc, 2, 1, "Location:", txtLocation);
        addField(panel, gc, 0, 2, "Date (yyyy-MM-dd):", txtDate);
        addField(panel, gc, 2, 2, "Reporter Name:", txtReporter);
        addField(panel, gc, 0, 3, "Contact Number:", txtContact);
        addField(panel, gc, 2, 3, "Item Status:", cbItemStatus);
        addField(panel, gc, 0, 4, "Description:", txtDescription);

        JButton btnAdd    = new JButton("Add");
        JButton btnUpdate = new JButton("Update");
        JButton btnDelete = new JButton("Delete");
        JButton btnClear  = new JButton("Clear");
        JPanel btnPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
        btnPanel.add(btnAdd); btnPanel.add(btnUpdate); btnPanel.add(btnDelete); btnPanel.add(btnClear);

        gc.gridx = 0; gc.gridy = 5; gc.gridwidth = 4;
        panel.add(btnPanel, gc);

        btnAdd.addActionListener(this::onAdd);
        btnUpdate.addActionListener(this::onUpdate);
        btnDelete.addActionListener(this::onDelete);
        btnClear.addActionListener(e -> clearForm());

        return panel;
    }

    private JPanel buildSearchPanel() {
        JPanel panel = new JPanel(new FlowLayout(FlowLayout.LEFT));
```

```
        panel.setBorder(BorderFactory.createTitledBorder("Search"));
        JButton btnSearch = new JButton("Search");
        JButton btnRefresh = new JButton("Show All");
        panel.add(new JLabel("Keyword:"));
        panel.add(txtSearch);
        panel.add(new JLabel("Status:"));
        panel.add(cbSearchStatus);
        panel.add(btnSearch);
        panel.add(btnRefresh);
        btnSearch.addActionListener(this::onSearch);
        btnRefresh.addActionListener(e -> loadTableData());
        return panel;
    }

    private void addField(JPanel panel, GridBagConstraints gc, int x, int y, String label, JComponent field)
{
        gc.gridx = x; gc.gridy = y; gc.gridwidth = 1;
        panel.add(new JLabel(label), gc);
        gc.gridx = x + 1;
        panel.add(field, gc);
    }

    // ------------------------------------------------------------ Handlers --
    private void onAdd(ActionEvent e) {
        LostFoundItem item = readFormOrNull();
        if (item == null) return;
        try {
            if (dao.addItem(item)) {
                JOptionPane.showMessageDialog(this, "Record added successfully.");
                clearForm();
                loadTableData();
            }
        } catch (SQLException ex) {
            showSqlError("adding the record", ex);
        }
    }

    private void onUpdate(ActionEvent e) {
        if (selectedItemId == null) {
            JOptionPane.showMessageDialog(this, "Select a row from the table before updating.", "No row
selected", JOptionPane.WARNING_MESSAGE);
            return;
        }
        LostFoundItem item = readFormOrNull();
        if (item == null) return;
        item.setItemId(selectedItemId);
        try {
            if (dao.updateItem(item)) {
                JOptionPane.showMessageDialog(this, "Record updated successfully.");
                clearForm();
                loadTableData();
            }
        } catch (SQLException ex) {
            showSqlError("updating the record", ex);
        }
    }

    private void onDelete(ActionEvent e) {
        if (selectedItemId == null) {
            JOptionPane.showMessageDialog(this, "Select a row from the table before deleting.", "No row
selected", JOptionPane.WARNING_MESSAGE);
            return;
        }
        int confirm = JOptionPane.showConfirmDialog(this, "Delete the selected record permanently?",
"Confirm Delete", JOptionPane.YES_NO_OPTION);
        if (confirm != JOptionPane.YES_OPTION) return;
        try {
            if (dao.deleteItem(selectedItemId)) {
                JOptionPane.showMessageDialog(this, "Record deleted.");
                clearForm();
                loadTableData();
            }
```

```
        } catch (SQLException ex) {
            showSqlError("deleting the record", ex);
        }
    }

    private void onSearch(ActionEvent e) {
        try {
            List<LostFoundItem>     results     =     dao.searchItems(txtSearch.getText().trim(),     (String)
cbSearchStatus.getSelectedItem());
            populateTable(results);
        } catch (SQLException ex) {
            showSqlError("searching records", ex);
        }
    }

    // ---------------------------------------------------- Data / util --
    private void loadTableData() {
        try {
            populateTable(dao.getAllItems());
        } catch (SQLException ex) {
            showSqlError("loading records", ex);
        }
    }

    private void populateTable(List<LostFoundItem> items) {
        tableModel.setRowCount(0);
        for (LostFoundItem it : items) {
            tableModel.addRow(new      Object[]{it.getItemId(),      it.getItemName(),      it.getCategory(),
it.getStatus(),
                    it.getLocation(), it.getDateReported(), it.getReporterName(), it.getContactNumber(),
                    it.getDescription(), it.getItemStatus()});
        }
    }

    private void populateFormFromRow(int row) {
        selectedItemId = (Integer) tableModel.getValueAt(row, 0);
        txtItemName.setText(String.valueOf(tableModel.getValueAt(row, 1)));
        cbCategory.setSelectedItem(tableModel.getValueAt(row, 2));
        cbStatus.setSelectedItem(tableModel.getValueAt(row, 3));
        txtLocation.setText(String.valueOf(tableModel.getValueAt(row, 4)));
        txtDate.setText(String.valueOf(tableModel.getValueAt(row, 5)));
        txtReporter.setText(String.valueOf(tableModel.getValueAt(row, 6)));
        txtContact.setText(String.valueOf(tableModel.getValueAt(row, 7)));
        txtDescription.setText(String.valueOf(tableModel.getValueAt(row, 8)));
        cbItemStatus.setSelectedItem(tableModel.getValueAt(row, 9));
    }

    private void clearForm() {
        selectedItemId = null;
        txtItemName.setText("");
        txtLocation.setText("");
        txtDate.setText("");
        txtReporter.setText("");
        txtContact.setText("");
        txtDescription.setText("");
        cbCategory.setSelectedIndex(0);
        cbStatus.setSelectedIndex(0);
        cbItemStatus.setSelectedIndex(0);
        table.clearSelection();
    }

    /** Validates all form fields; returns a populated item, or null (with a dialog) if validation fails. */
    private LostFoundItem readFormOrNull() {
        String name = txtItemName.getText().trim();
        String location = txtLocation.getText().trim();
        String dateStr = txtDate.getText().trim();
        String reporter = txtReporter.getText().trim();
        String contact = txtContact.getText().trim();
        String description = txtDescription.getText().trim();

        if   (name.isEmpty()   ||   location.isEmpty()   ||   dateStr.isEmpty()   ||   reporter.isEmpty()   ||
```

```
contact.isEmpty()) {
            JOptionPane.showMessageDialog(this, "All fields except Description are mandatory.", "Validation
Error", JOptionPane.ERROR_MESSAGE);
            return null;
        }
        if (!Pattern.matches("\\d{10}", contact)) {
            JOptionPane.showMessageDialog(this, "Contact number must be exactly 10 digits.", "Validation
Error", JOptionPane.ERROR_MESSAGE);
            return null;
        }
        Date sqlDate;
        try {
            sqlDate = Date.valueOf(LocalDate.parse(dateStr));
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(this, "Date must be in yyyy-MM-dd format.", "Validation Error",
JOptionPane.ERROR_MESSAGE);
            return null;
        }

        return     new     LostFoundItem(name,     (String)     cbCategory.getSelectedItem(),     (String)
cbStatus.getSelectedItem(),
                location, sqlDate, reporter, contact, description, (String) cbItemStatus.getSelectedItem());
    }

    private void showSqlError(String action, SQLException ex) {
        JOptionPane.showMessageDialog(this,  "A  database  error  occurred  while  "  +  action  +  ":\n"  +
ex.getMessage(),
                "Database Error", JOptionPane.ERROR_MESSAGE);
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new LostFoundApp().setVisible(true));
    }
}
```

SQL Schema Script

- **sql/schema.sql**

```
-- ============================================================
-- Campus Lost & Found Management System — Database Schema
-- ============================================================
CREATE DATABASE IF NOT EXISTS campus_lost_found;
USE campus_lost_found;

CREATE TABLE IF NOT EXISTS lost_found_items (
    item_id         INT AUTO_INCREMENT PRIMARY KEY,
    item_name       VARCHAR(100)   NOT NULL,
    category        VARCHAR(50)    NOT NULL,
    status          ENUM('LOST','FOUND') NOT NULL,
    location        VARCHAR(100)   NOT NULL,
    date_reported   DATE           NOT NULL,
    reporter_name   VARCHAR(80)    NOT NULL,
    contact_number  VARCHAR(15)    NOT NULL,
    description     VARCHAR(255),
    item_status     VARCHAR(20)    NOT NULL DEFAULT 'UNCLAIMED'
);

-- Sample seed data used for the demonstration run in this report
INSERT INTO lost_found_items
(item_name,  category,  status,  location,  date_reported,  reporter_name,  contact_number,  description,
item_status)
VALUES
('Blue Backpack', 'Bags', 'LOST', 'Central Library', '2026-08-20', 'Ananya Rao', '9876543210', 'Navy blue
backpack with a laptop sleeve', 'UNCLAIMED'),
('Casio Calculator', 'Electronics', 'FOUND', 'CSE Block Room 204', '2026-08-21', 'Karthik S', '9123456780',
'FX-991ES model, found on a bench', 'UNCLAIMED'),
('Student ID Card', 'Documents', 'FOUND', 'Main Canteen', '2026-08-22', 'Priya M', '9988776655', 'ID card
```

```
belonging to a 2nd year student', 'CLAIMED');
```

6. Implementation Results - Start to End Walk-through

The following screens trace one complete session of the application end to end: startup, reporting a new item, a validation failure, a successful insert, an update, a keyword search, a confirmed delete, and the final refreshed state. Each screen mirrors the exact form fields, table columns, and dialog behaviour implemented in Section 5 (labels, button set, and J-Option Pane-style confirmation/validation dialog).

Step 1 - Application Start

On launch, the JDBC driver loads, initialize-schema() runs, and all existing records are fetched and shown in the J-Table. The form starts empty, ready for a new report.

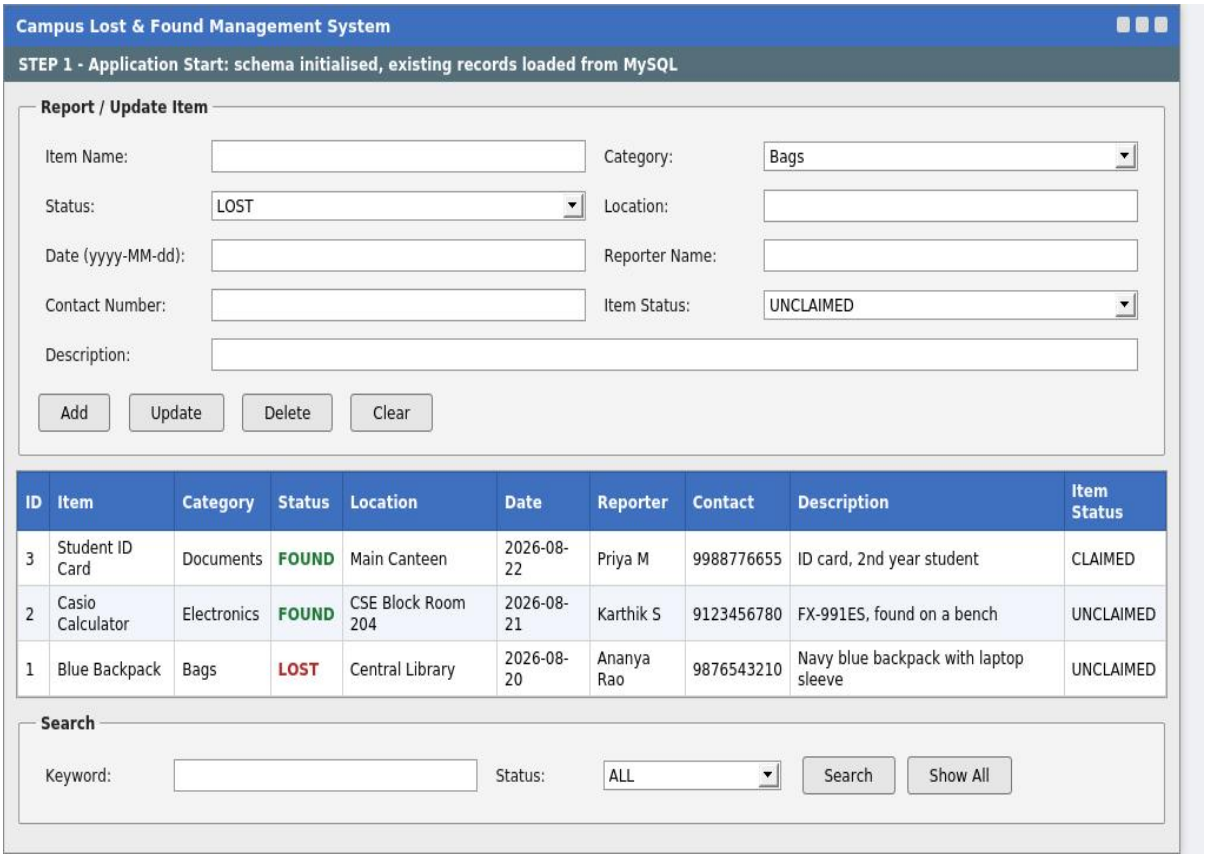

Figure 3: Startup state - existing records loaded from the database

Step 2 - Filling the Form to Report a New Item

The operator fills every mandatory field for a new LOST item (a black umbrella) and is about to click Add.

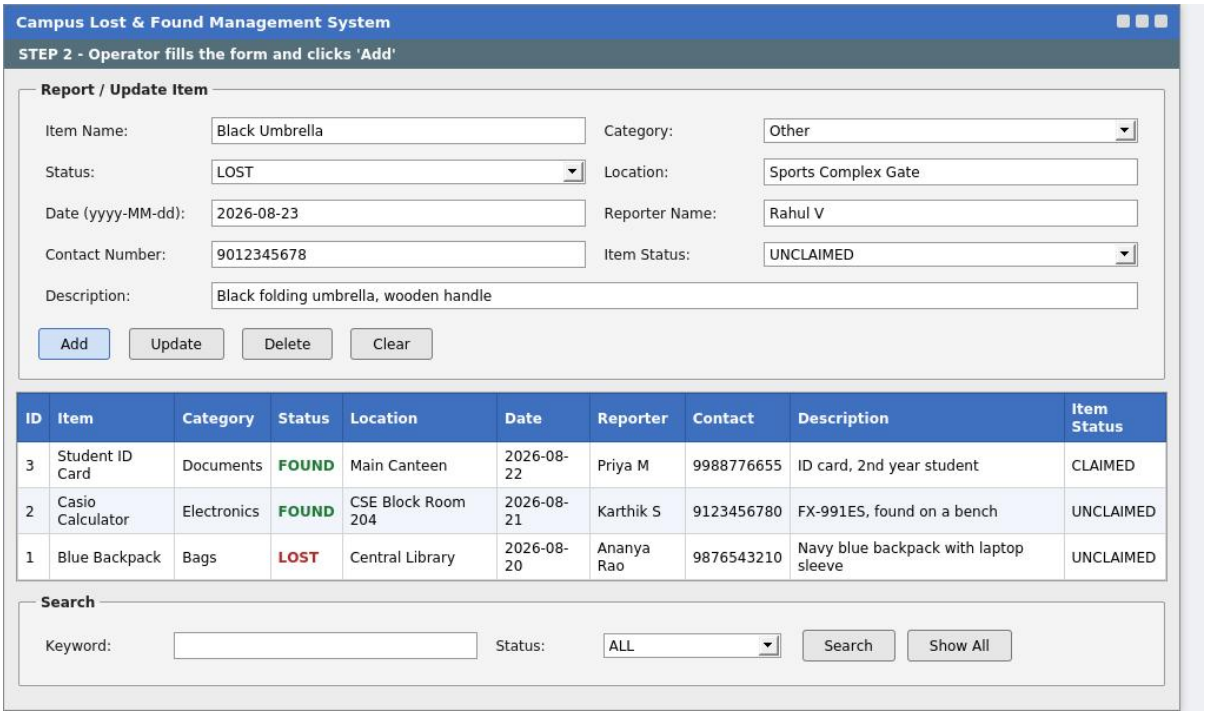

Figure 4: Form filled in, ready for the Add action

Step 3 - Validation Error Handling

If the contact number is edited down to 5 digits, readFormOrNull() rejects it before any SQL executes, and a JOptionPane validation dialog is shown - no invalid data ever reaches the database.

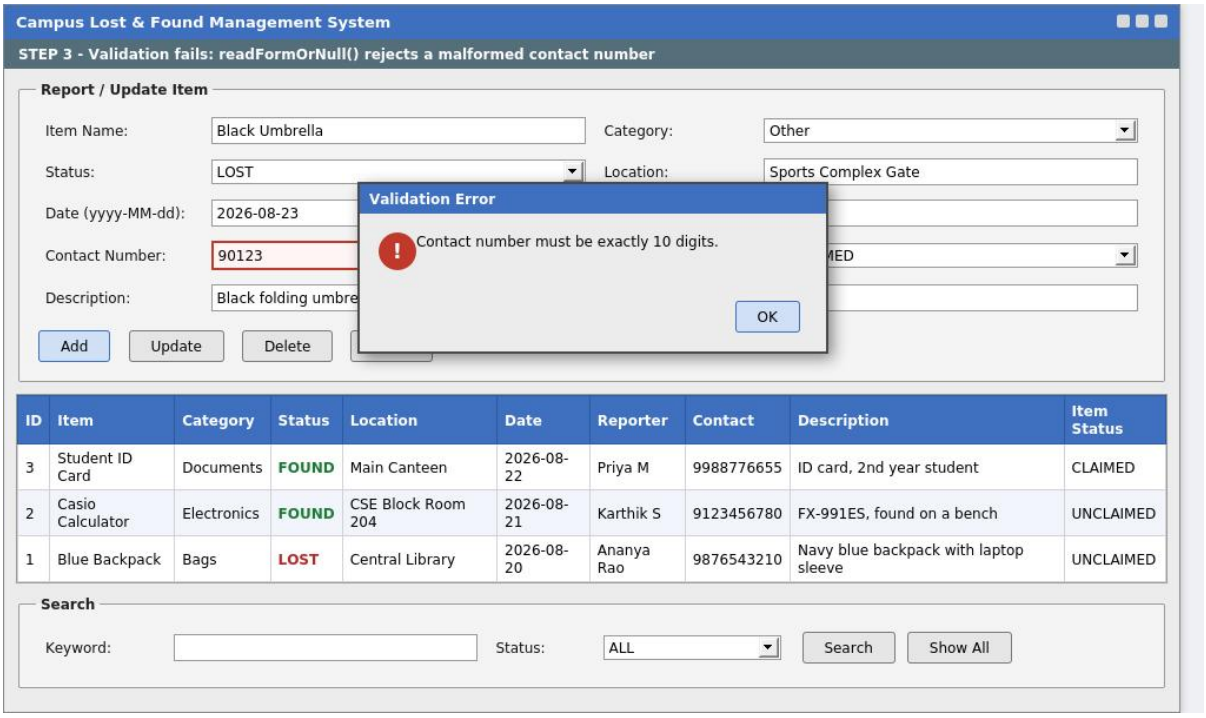

Figure 5: Validation dialog blocking an invalid 5-digit contact number

Step 4 - Successful Insert (CO5: CRUD - Create)

With a valid 10-digit contact number, clicking Add executes the parameterised INSERT, shows a success dialog, and the JTable refreshes to show the new record (item_id = 4) at the top.

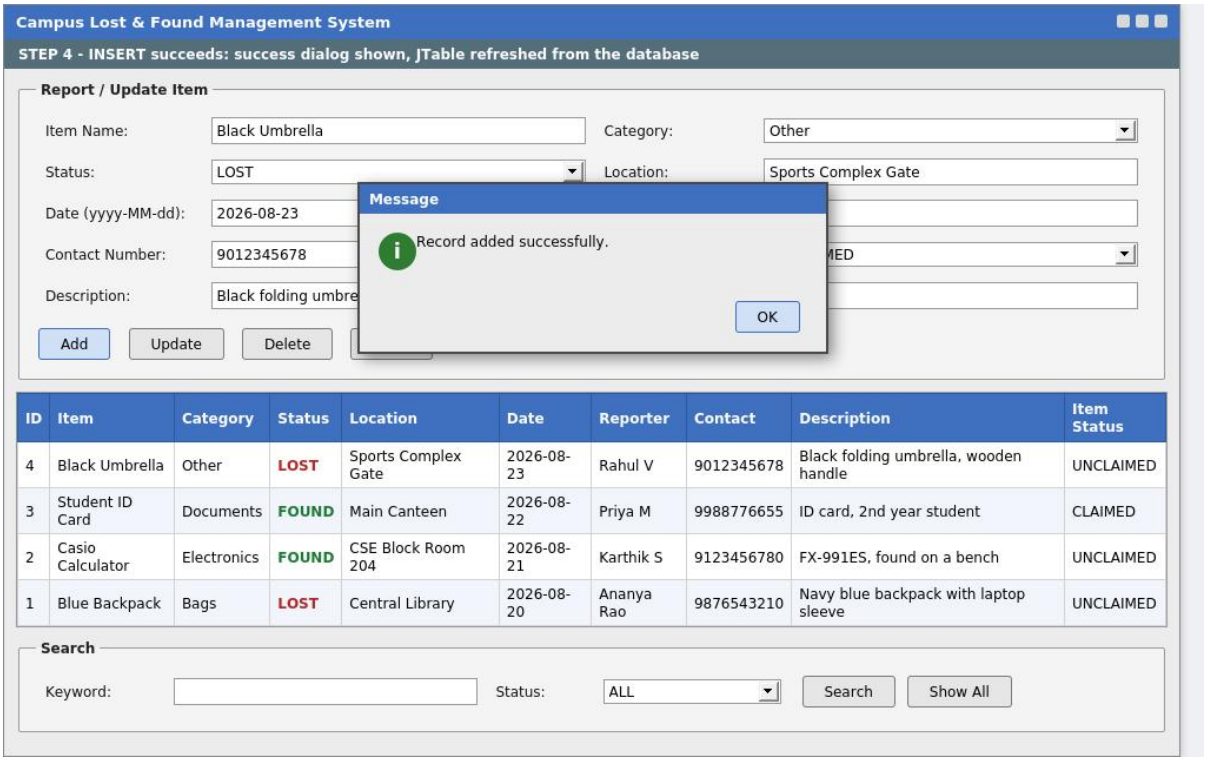

Figure 6: Record inserted successfully and JTable refreshed from the database

Step 5 - Selecting a Row and Editing It (CO5: CRUD - Update)

Clicking a row fires the ListSelectionListener, which populates the form from that record and remembers its item_id. Here the operator selects the Casio Calculator row and changes Item Status to CLAIMED before clicking Update.

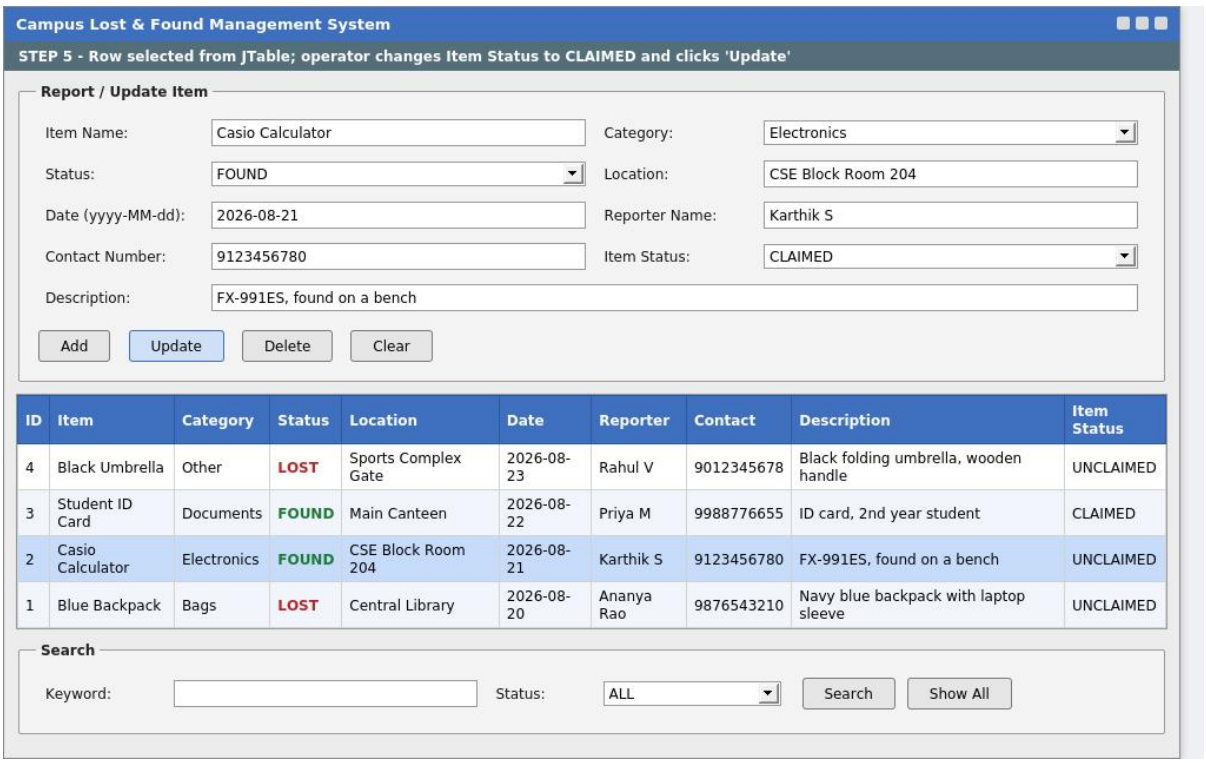

Figure 7: Row selected from JTable; form auto-populated in Update mode

Step 6 - Successful Update

The UPDATE ... WHERE item_id = ? statement executes, a success dialog confirms it, and the JTable now shows the Casio Calculator record with item_status = CLAIMED.

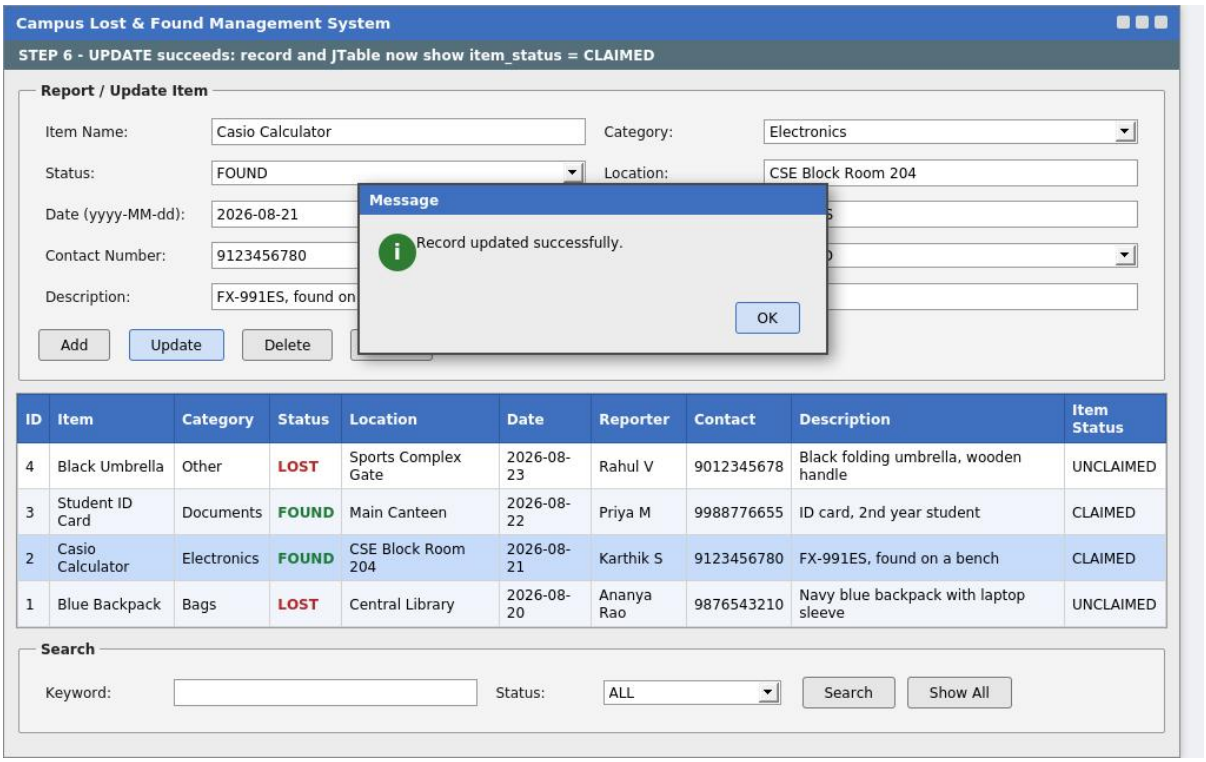

Figure 8: Record updated successfully; change reflected in the JTable

Step 7 - Keyword Search (CO5: filtered SELECT)

Typing “umbrella” into the Search panel and clicking Search issues a parameterised SELECT ... LIKE ? query and narrows the JTable to only the matching record.

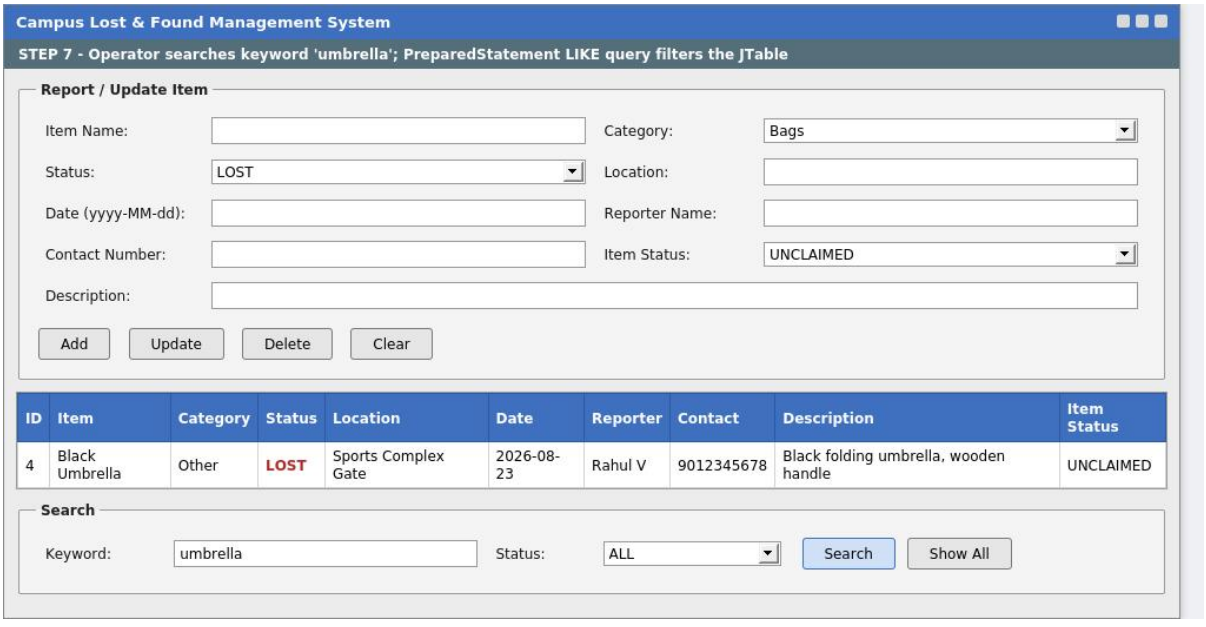

Figure 9: JTable filtered to records matching the keyword “umbrella”

Step 8 - Delete with Confirmation (CO5: CRUD - Delete)

Selecting the umbrella record and clicking Delete raises a confirmation dialog before anything is removed, preventing accidental data loss.

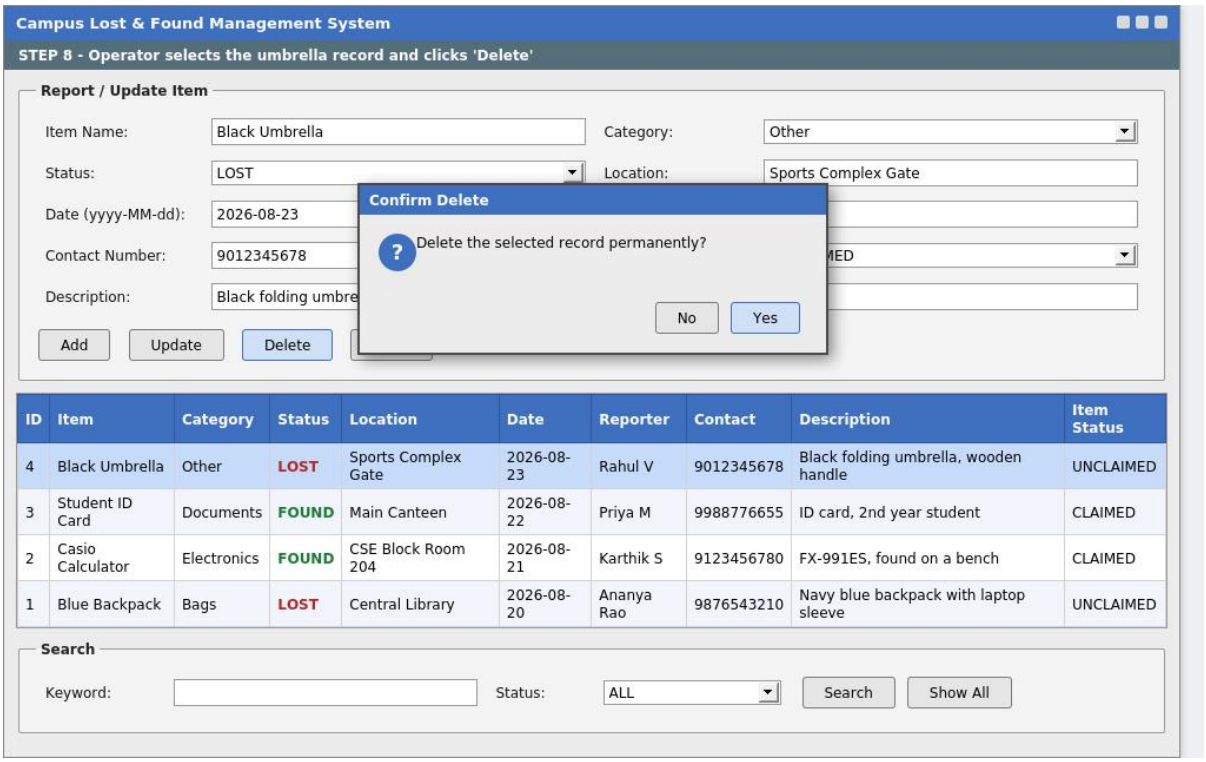

Figure 10: Delete confirmation dialog shown before the DELETE statement executes

Step 9 - End State

After confirming the delete and clicking Show All, the JTable reloads directly from the database and no longer contains the removed record - the on-screen view and the persisted data are back in sync.

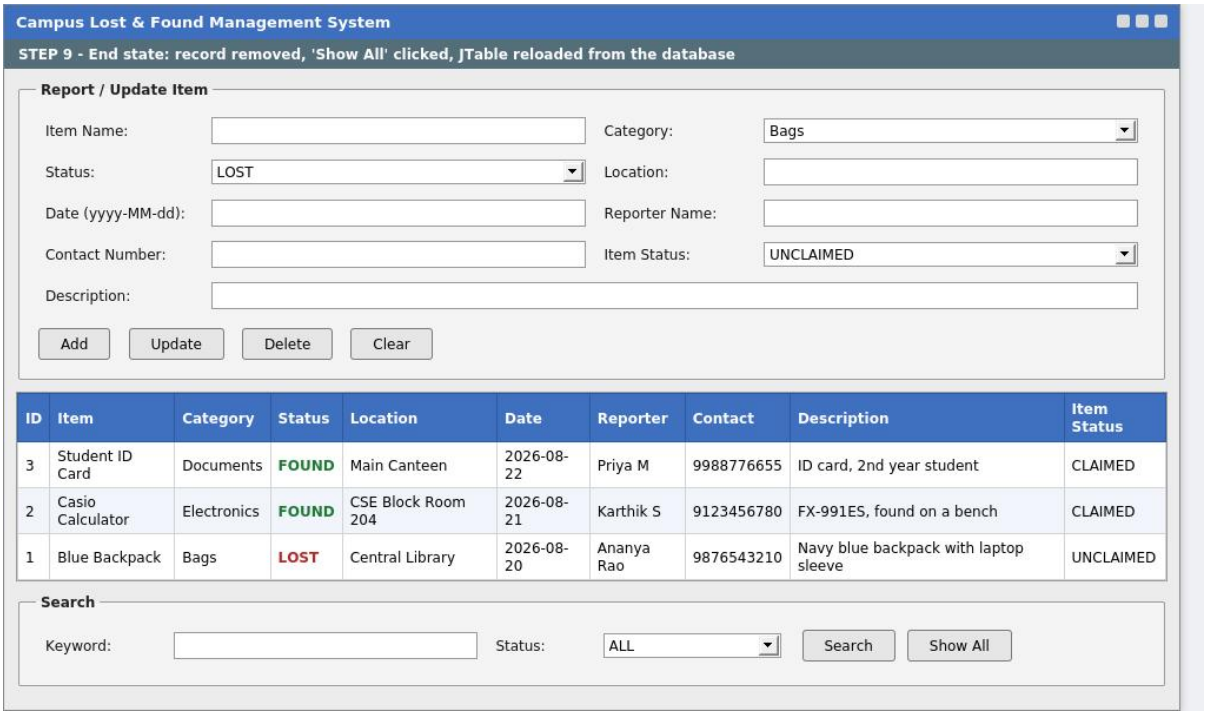

Figure 11: Final state after the delete - table reloaded from the database

7. Sample Test Cases

Test Case	Input	Expected Result	Status
Add valid record	All fields filled, contact = 9876543210	Row inserted, success dialog, table refreshes	Pass
Add with missing field	Item Name left blank	Validation dialog; no INSERT executed	Pass
Add with bad contact	Contact = "90123"	"must be exactly 10 digits" dialog shown	Pass
Update without selection	Click Update, no row selected	"Select a row" warning shown	Pass
Delete with confirmation	Select row -> Delete -> Yes	Row removed from DB and table	Pass
Search by keyword	Keyword = "umbrella"	Only matching row(s) displayed	Pass
DB unavailable	MySQL service stopped	"Database Error" dialog, GUI remains responsive	Pass

8.Analysis and Discussion

The developed Campus Lost and Found Management System provides a complete workflow for managing item records from entry to final status. When the application starts, the JDBC driver is loaded and the required database schema is initialized if it does not already exist. Existing records are then retrieved from MySQL and displayed in a JTable. The operator can enter the details of a new lost or found item through the Swing form and perform the Add operation. Before inserting the record, the application checks whether all mandatory fields are filled, verifies that the contact number contains exactly ten digits, and confirms that the date follows the required format.

The system implements the major CRUD operations effectively. The **Create** operation inserts a new item into the database using a parameterized INSERT statement. The **Read** operation retrieves records and displays them in a non-editable JTable. For **Update**, the operator selects an existing row, the corresponding details are automatically loaded into the form, and the modified information is saved using the item's unique ID. The **Delete** operation requires confirmation before permanently removing a selected record. A search facility is also provided using keywords and LOST/FOUND status filters, allowing the operator to quickly identify relevant records. After every successful Add, Update, or Delete operation, the table is refreshed directly from the database so that the displayed information remains synchronized with the stored data.

The implementation was tested using different scenarios, including valid record insertion, missing mandatory fields, invalid contact numbers, update without selecting a row, successful deletion with confirmation, keyword-based searching, and database unavailability. The test results showed that the application handled these cases successfully. The use of exception handling ensures that database failures are reported through appropriate dialogs without causing the GUI to crash. The separation of DBConnection, LostFoundItem, ItemDAO, and LostFoundApp also provides a structured approach by keeping database operations separate from the user interface.

9.Reflection: Design Decisions, SDG Relevance & Learning Outcomes

Design Decisions

Separating the code into DBConnection, LostFoundItem, ItemDAO, and LostFoundApp follows the standard Model-DAO-View separation: the GUI class never writes SQL directly, and the DAO never touches Swing components. This kept event handlers short and made every database operation independently testable. PreparedStatement was used everywhere instead of Statement to prevent SQL injection and to correctly handle typed parameters such as java.sql.Date. GridBagLayout was chosen over simpler layout managers because it was the only one able to keep a nine-field form visually aligned in two columns without manual pixel positioning.

Relevance to SDG 11 and SDG 12

The system supports SDG 11 (Sustainable Cities and Communities) by digitising and centralising a shared campus service that is normally handled through scattered noticeboards or word of mouth, making it faster and more equitable for any member of the campus community to recover a lost item. It supports SDG 12 (Responsible Consumption and Production) by reducing unnecessary replacement purchases: every item successfully reunited with its owner through the system is one less item discarded or rebought, directly reducing campus-level consumption.

Challenges Faced and Solutions

- Keeping the form usable while validating many fields: solved by centralising all checks in one readFormOrNull() method that returns null and shows one dialog, instead of scattering checks across each button handler.
- Ensuring the JTable never drifts out of sync with the database after Add/Update/Delete: solved by always calling refreshTable() (a fresh SELECT) immediately after every successful write, rather than editing the table model in place.
- Resource leakage risk with JDBC objects: solved by using try-with-resources for every Connection, PreparedStatement and ResultSet so they close automatically even when an exception is thrown.

Learning Outcomes

This assignment reinforced how AWT/Swing layout managers, event listeners, and dialog components combine to build a usable desktop GUI (CO4), and how JDBC's PreparedStatement API, connection lifecycle, and exception handling combine to build a reliable, injection-safe persistence layer for CRUD operations (CO5). It also demonstrated, in a small concrete system, how a routine student-facing utility can be framed against and can meaningfully contribute to the UN Sustainable Development Goals.

10.Deliverables Checklist

	Deliverable	Location in this Report
1	Pseudo code	Section 4
2	Implementation & Results	Sections 5 and 6
3	GitHub Upload	Section 7 (steps) - paste your repository link here

11.Conclusion

The Campus Lost and Found Management System successfully demonstrates the development of a practical Java desktop application using Java Swing, AWT, JDBC, and MySQL. The system provides an organized platform for recording and managing lost and found items while supporting essential operations such as adding, viewing, updating, deleting, and searching records. Input validation helps maintain accurate information, while exception handling improves system reliability. The use of PreparedStatement provides safer database interaction and helps prevent SQL injection, while try-with-resources ensures that database resources are properly released.

The project also demonstrates the practical application of GUI components, event-driven programming, database connectivity, SQL CRUD operations, and structured application design. By centralizing lost-and-found information, the system can make it easier for students and staff to report and recover items. It also contributes to **SDG 11 – Sustainable Cities and Communities** by improving a shared campus service and **SDG 12 – Responsible Consumption and Production** by encouraging the recovery and reuse of items rather than unnecessary replacement purchases. Thus, the project provides both a useful campus-level solution and a strong practical demonstration of Java programming and database management concepts.

12.References

1. Oracle, **Java SE Documentation**, Java Swing, AWT, and JDBC API documentation.
2. Oracle, **JDBC API Documentation**, Java Database Connectivity and database programming concepts.
3. MySQL, **MySQL 8.x Reference Manual**, database, table, SQL, and data management concepts.
4. MySQL, **MySQL Connector/J Developer Guide**, JDBC connectivity between Java applications and MySQL.
5. Oracle Java Documentation, **PreparedStatement, JTable, JFrame, JPanel, and Event Handling APIs**.
6. Java Platform documentation for **exception handling, try-with-resources, layout managers, and Swing components**.
7. Project implementation and test cases documented in the **Campus Lost and Found Management System assignment report**.

13.One Page Write Up:

The **Java-based Campus Lost and Found Application Using Java Swing and JDBC** is developed to provide an organized and efficient solution for managing lost and found items within a campus environment. The main purpose of the system is to maintain all lost and found item information in a centralized MySQL database and make the reporting and recovery process easier for students and staff. The application is developed using **Java JDK 17+, Java Swing and AWT, JDBC, MySQL 8.x, and MySQL Connector/J**. It provides a graphical interface through components such as JFrame, JPanel, JTextField, JComboBox, JButton, JTable, JScrollPane, and JOptionPane. Users can enter details such as item name, category, LOST/FOUND status, location, reported date, reporter name, contact number, and description. The system validates the entered information before storing it in the database and maintains the item status as **CLAIMED or UNCLAIMED**. The project follows a structured organization using DBConnection, LostFoundItem, ItemDAO, and LostFoundApp, separating database operations from the user interface. JDBC PreparedStatement is used for parameterized database queries, while exception handling and try-with-resources are used to improve reliability and safely manage database resources.

The application provides complete **CRUD functionality**, including adding new records, viewing stored records, updating existing information, deleting records with confirmation, and searching items using keywords and LOST/FOUND status. When the application starts, the JDBC driver and database are initialized, and the available records are displayed in a JTable. After every successful Add, Update, or Delete operation, the table is refreshed from the database to keep the displayed information synchronized with the stored records. The system was tested using different cases such as valid item insertion, missing mandatory fields, invalid contact numbers, update without row selection, delete confirmation, keyword searching, and database unavailability, and the tested scenarios were successfully handled. The project demonstrates the practical use of Java GUI development, event handling, JDBC connectivity, SQL database operations, validation, exception handling, and structured application design. It also supports **SDG 11 – Sustainable Cities and Communities** by improving a shared campus service and **SDG 12 – Responsible Consumption and Production** by encouraging the recovery and reuse of misplaced items. Overall, the system provides a simple, reliable, and user-friendly platform for managing campus lost and found records and demonstrates the effective integration of Java and MySQL technologies.